

MAJOR PROJECT

PROJECT TITLE :

Implementation of Auto scaling Web servers with application Load Balancer using AWS.

SUBMITTED BY : Lokeshwar Reddy kukkala

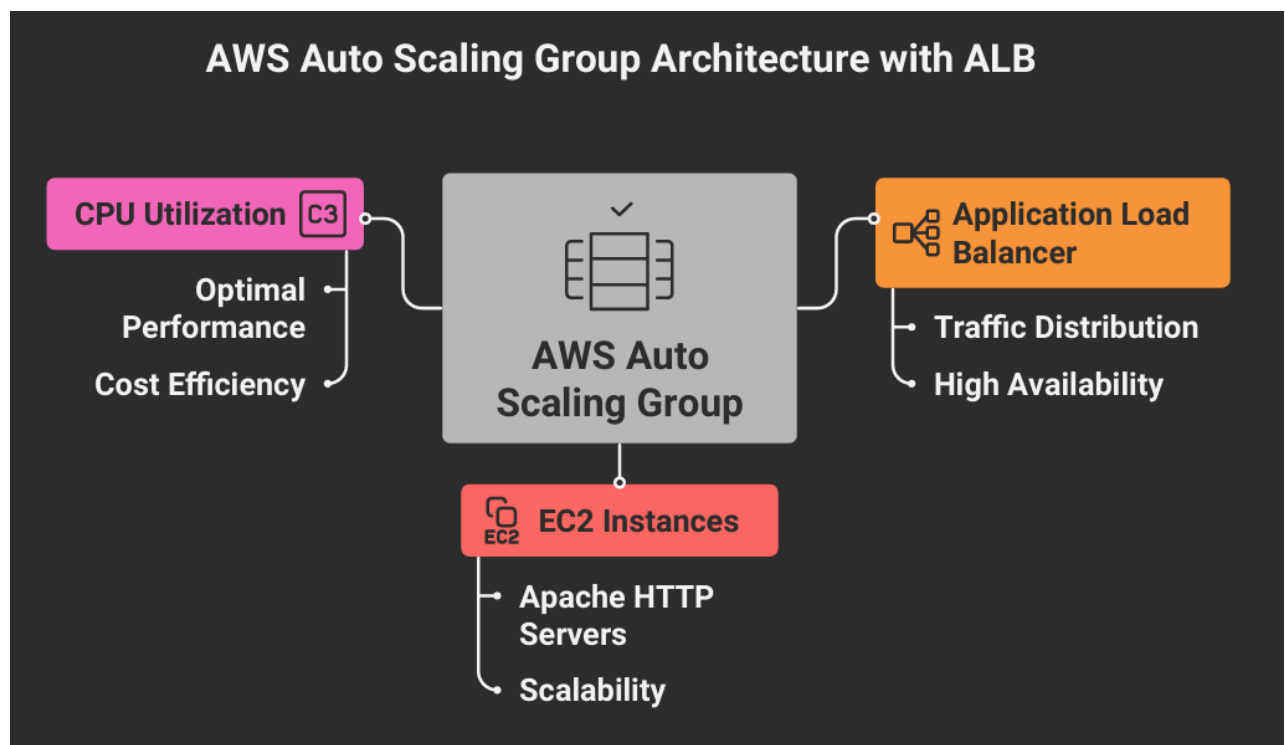
DOMAIN : AWS Cloud computing

SUBMITTED TO : SkillForge Company

DATE OF SUBMISSION : 01/12/2025

INTRODUCTION :

Cloud computing enables applications to be highly available and scalable. In this project, an AWS Auto Scaling Group is created with a minimum of 1 and a maximum of 3 EC2 instances. An HTTP web server is automatically installed on each instance using the User Data feature. A target tracking scaling policy with 50% CPU utilization is configured, and an Application Load Balancer is integrated with the Auto Scaling Group to distribute traffic efficiently. This project demonstrates scalability, availability, and load balancing in AWS Cloud.



OBJECTIVES OF PROJECT :

- To create an Auto scaling Group with defined minimum and maximum capacity.
- To automatically install HTTP server on all EC2 instances using User Data.
- To configure a Dynamic Target Tracking Scaling Policy based on CPU usage.
- To create an Application Load Balancer for traffic distribution.
- To register the Auto Scaling Group with the Target Group of the Load Balancer.
- To verify automatic traffic distribution using the ALB DNS endpoint.

TOOLS AND SERVICES USED :

1. Amazon EC2
2. Launch Template
3. Auto Scaling Group (ASG)
4. Application Load Balancer (ALB)
5. Target Group
6. Security Group (SG)
7. HTTP Server (Apache)

SYSTEM ARCHITECTURE OVERVIEW :

- Users send request to the Application Load Balancer (ALB).
- ALB forwards the traffic to healthy EC2 instances inside the Auto Scaling Group.
- Auto Scaling Group automatically adds or removes instances based on CPU utilization.
- Each EC2 instance runs an HTTP web server installed using User Data.

STEP-BY-STEP IMPLEMENTATION :

STEP 1 : CREATING A LAUNCH TEMPLATE

- A Launch Template was created with :
 - AMI : Amazon Linux
 - Instance Type : t2.micro
 - Key pair : No key pair
 - Security Group : Allowed HTTP (port 80) and SSH (port 22)

Launch Templates (1)

Info

Q Search

Launch Template ID

▼

Launch Template Name

▼

Default Version

▼

Latest Version

▼

Create Time

▼

Created By

lt-00ce655554045c4eb

major-template

1

1

2025-11-30T09:14:42.000Z

arn:aws:iam::539609

1

Create launch template

- Launch Template name : major-template

Launch template version details

Actions

Delete template version

Version 1 (Default)	Description -	Date created  2025-11-30T09:14:42.000Z	Created by  arn:aws:iam::539609142189:root
Instance details	Storage	Resource tags	Network interfaces
Advanced details			
AMI ID  ami-0fa3fe0fa7920f68e	Instance type  t2.micro	Availability Zone -	Availability Zone Id -
Key pair name -	Security groups -	Security group IDs  sg-0dab3d93bf3a164c5	

User data



```
#!/bin/bash
#use this for your user data (script from top to bottom)
#install httpd (Linux 2 version)
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
echo "<h1>This is my Auto scaling Group from $(hostname -f)</h1>" > /var/www/html/index.html
```

The script automatically installs and starts the Apache HTTP server on every EC2 instance created by Auto Scaling.

STEP 2 : CREATING AUTO SCALING GROUP

- Auto Scaling Group was created using the Launch Template.
- Configuration :
 - Minimum Capacity : 1 Instance
 - Desired Capacity : 2 Instance
 - Maximum Capacity : 3 Instance

VPC and subnets were selected for proper networking.

Auto Scaling groups (1) Info

Last updated less than a minute ago

Launch configurations

Launch templates ↗

Actions ▼

Create Auto Scaling group

Q

Search your Auto Scaling groups

<

1

>

⚙

☐	Name	Launch template/configuration ↗	Instances ▼	Status ▼	Desired capacity ▼	Min ▼	Max
☐	my-final-asg	major-template Version Default	2	-	2	1	3

- Auto Scaling Group Name : my-final-group

my-final-asg

my-final-asg Capacity overview Edit

arm:aws:autoscaling:us-east-1:539609142189:autoScalingGroup:abe79168-9e6d-4e76-9a78-bd4ccc534d84:autoScalingGroupName/my-final-asg

Desired capacity	Scaling limits (Min - Max)	Desired capacity type	Status
2	1 - 3	Units (number of instances)	-

Date created

Sun Nov 30 2025 14:48:33 GMT+0530 (India Standard Time)

- Availiable Networks :

Network		
Availability Zones use1-az2 (us-east-1b) use1-az3 (us-east-1e) use1-az1 (us-east-1a)	Subnet ID 📄 subnet-01bef03e253dfdea5 📄 subnet-04a863a046d0f310f 📄 subnet-04f2aa022cd72a454	Availability Zone distribution Balanced best effort

Activity history (3)

Q

Filter activity history

<

1

>

⚙

Status	Description	Cause	Start time	End time
Successful	Updating load balancers/target groups: Successful. Status Reason: Added: arm:aws:elasticloadbalancing:us-east-1:539609142189:target-group/my-target/232a260ad683c539 (Target Group).		2025 November 30, 03:30:51 PM +05:30	2025 November 30, 03:30:52 PM +05:30
Successful	Launching a new EC2 instance: i-0b88aa31a0070b2cf	At 2025-11-30T09:18:33Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 2. At 2025-11-30T09:18:34Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 2.	2025 November 30, 02:48:36 PM +05:30	2025 November 30, 02:48:52 PM +05:30
Successful	Launching a new EC2 instance: i-0e624045408a72298	At 2025-11-30T09:18:33Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 2. At 2025-11-30T09:18:34Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 2.	2025 November 30, 02:48:35 PM +05:30	2025 November 30, 02:49:07 PM +05:30

Load balancing

Load balancer target groups
my-target

Classic Load Balancers

-

- In Auto Scaling the Load Balancer was attached along with “ Target Group “.

STEP 3 : CREATING TARGET TRACKING SCALING POLICY

- A Dynamic Target Tracking Scaling Policy was created.
- Metric used : Average CPU Utilization.'
- Target value set to : 50%
- When CPU usage increases above 50%, new instance are launched.
- When CPU usage decreases, extra instances are terminated automatically.

STEP 4 : CREATING TARGET GROUP

- A Target Group was created with :
 - Target type : Instance
 - Protocol : HTTP
 - Port : 80
 - Health path : /

- Auto Scaling Group was attached to this Target Group.

Target groups (1) [Info](#) [What's new?](#)

Q Filter target groups

☐	Name	ARN	Port	Protocol	Target type	Load balancer	VPC ID
☐	my-target	arn:aws:elasticloadbalancing:us-east-1:539609142189:targetgroup/my-target/232a260ad683c539	80	HTTP	Instance	my-load-balancer	vpc-02b49e...

- Target Group Name : my-target

my-target [Actions](#)

Details

[arn:aws:elasticloadbalancing:us-east-1:539609142189:targetgroup/my-target/232a260ad683c539](#)

Target type Instance	Protocol : Port HTTP: 80	Protocol version HTTP1	VPC vpc-02b49ae05dd92ced
IP address type IPv4	Load balancer my-load-balancer		

- Target Instances : 2 Instanes

Registered targets (2) [Info](#) [Anomaly mitigation: Not applicable](#) [Deregister](#) [Register targets](#)

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Q Filter targets

☐	Instance ID	Name	Port	Zone	Health status	Health status details	Admini...	Overri..
☐	i-Oe624045408a72298	web server1	80	us-east-1a (us...)	Healthy	-	No override.	No over
☐	i-0b88aa31a0070b2cf	web server 2	80	us-east-1b (us...)	Healthy	-	No override.	No over

- Were Target Group are attached with the Load Balancer.

STEP 5 : CREATING APPLICATION LOAD BALANCER

- An Application Load Balancer was created.

- Configuration :
 - Scheme : Internet-facing
 - Listener : HTTP on Port 80
 - Subnets : Selected from different Availability Zones.
 - Target group : my-target
- The Target Group created earlier was attached to the ALB listener.

Load balancers (1) [What's new?](#)

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Q Filter load balancers

☐	Name	State	Type	Scheme	IP address type	VPC ID	Availability Zones
☐	my-load-balancer	Active	application	Internet-facing	IPv4	vpc-02b49aee05dd92ced	3 Availability Zones

- Load Balancer Name : my-load-balancer

my-load-balancer

Details

Load balancer type Application	Status Active	VPC vpc-02b49aee05dd92ced	Load balancer IP address type IPv4
Scheme Internet-facing	Hosted zone Z35SXDOTRQ7X7K	Availability Zones subnet-01bef03e253dfdea5 us-east-1b (use1-az2) subnet-04f2aa022cd72a454 us-east-1a (use1-az1) subnet-00f111c0a2fde3e96 us-east-1c (use1-az4)	Date created November 30, 2025, 15:09 (UTC+05:30)

Load balancer ARN arn:aws:elasticloadbalancing:us-east-1:539609142189:loadbalancer/app/my-load-balancer/db2de5b4d364334e	DNS name my-load-balancer-2127080806.us-east-1.elb.amazonaws.com (A Record)
--	---

Listeners and rules (1) [Info](#)

A listener checks for connection requests on its configured protocol and port. Traffic received by the listener is routed according to the default action and any additional rules.

Q Filter listeners

☐	Protocol:Port	Default action	Rules	ARN	Security policy	Default SSL/TLS cer
☐	HTTP:80	• Forward to target group my-target : 1 (100%) Target group stickiness: Off	1 rule	☒ ARN	Not applicable	Not applicable

STEP 6 : REGISTERING AUTO SCALING GROUP WITH LOAD BALANCER

- Auto Scaling Group was attached to the ALB Targer Group.
- All EC2 instances created by ASG were automatically registered as targets.
- Health checks were enabled to monitor instance status.

STEP 7 : SECURITY GROUPS

The Allowed ports in the security group are :

- SSH (22) for remote access.
- HTTP (80) for web traffic.
- Security Group Name : my-security-group

Details

Security group name  my-security-group	Security group ID  sg-0dab3d93bf3a164c5	Description  Allow ssh access to developers	VPC ID vpc-02b49aee05dd92ced 
Owner  539609142189	Inbound rules count 2 Permission entries	Outbound rules count 1 Permission entry	

sg-0dab3d93bf3a164c5 - my-security-group



Inbound rules (2)							
Search							
☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	
☐	-	sgr-0273021aee6806c2	IPv4	HTTP	TCP	80	
☐	-	sgr-068b8aa0588be9662	IPv4	SSH	TCP	22	

TESTING THE LOAD BALANCER :

➤ INSTANCES :

- The Load balancer were created two EC2 instances namely :
 - web server 1
 - web server 2

Instances (2) [Info](#) [Refresh](#) [Connect](#) [Instance state](#) [Actions](#) [Launch instances](#)

[All states](#) [< 1 >](#) [Settings](#)

☐	Name ↗	Instance ID	Instance state ↕	Instance type ↕	Status check	Alarm status	Availability Zone ↕	Public IPv4
☐	web server1	i-0e624045408a72298	Running 🔍 🔍	t2.micro	2/2 checks passed	View alarms +	us-east-1a	ec2-3-236-36-98.compute-1.amazonaws.com
☐	web server 2	i-0b88aa31a0070b2cf	Running 🔍 🔍	t2.micro	2/2 checks passed	View alarms +	us-east-1b	ec2-44-211-135-167.compute-1.amazonaws.com

Web server 1 :

Instance summary for i-0e624045408a72298 (web server1) [Info](#) [Refresh](#) [Connect](#) [Instance state](#) [Actions](#)

Updated less than a minute ago

Instance ID i-0e624045408a72298	Public IPv4 address 3.236.36.98 open address	Private IPv4 addresses 172.31.11.94
IPv6 address -	Instance state Running	Public DNS ec2-3-236-36-98.compute-1.amazonaws.com open address

Web server 2 :

Instance summary for i-0b88aa31a0070b2cf (web server 2) [Info](#) [Refresh](#) [Connect](#) [Instance state](#) [Actions](#)

Updated less than a minute ago

Instance ID i-0b88aa31a0070b2cf	Public IPv4 address 44.211.135.167 open address	Private IPv4 addresses 172.31.89.227
IPv6 address -	Instance state Running	Public DNS ec2-44-211-135-167.compute-1.amazonaws.com open address

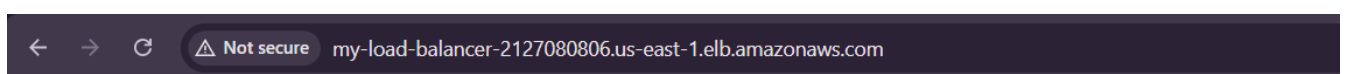
- The DNS name of the ALB was copied.
- This DNS endpoint was opened in a web browser.

- The web page displayed successful output from EC2 instances.
- Traffic was automatically distributed among instances.
- When load increased, new instances were automatically launched.
- When load decreased, instances were terminated automatically.

my-load-balancer Actions			
▼ Details			
Load balancer type Application	Status Active	VPC vpc-02b49aee05dd92ced	Load balancer IP address type IPv4
Scheme Internet-facing	Hosted zone Z35SXDOTRQ7X7K	Availability Zones subnet-01bef03e253dfdea5 us-east-1b (use1-az2) subnet-04f2aa022cd72a454 us-east-1a (use1-az1) subnet-00f111c0a2fde3e96 us-east-1c (use1-az4)	Date created November 30, 2025, 15:09 (UTC+05:30)
Load balancer ARN arn:aws:elasticloadbalancing:us-east-1:539609142189:loadbalancer/app/my-load-balancer/db2de5b4d364334e		DNS name Info my-load-balancer-2127080806.us-east-1.elb.amazonaws.com (A Record)	

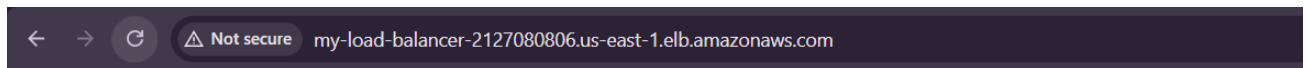
OUTPUT :

- HTTP server installed automatically on all instances.
- Users were able to access the application using the ALB DNS name without knowing the internal EC2 IP address.



This is my Auto scaling Group from ip-172-31-11-94.ec2.internal

- The above Apache web server is from the first instances (web server 1).
- When an user refresh the browser it will serve the HTTP request to another server.



This is my Auto scaling Group from ip-172-31-89-227.ec2.internal

- This is the second server namely web server 2 .

CONCLUSION :

In this project, a fully automated, scalable, and highly available web infrastructure was successfully implemented using AWS services. The Auto Scaling Group automatically managed server capacity based on CPU utilization, while the Application Load Balancer efficiently distributed incoming traffic across multiple EC2 instances. This project provides real-world experience in designing cloud-based scalable architectures and demonstrates the power of AWS cloud computing in handling dynamic workloads efficiently.